C LANGUAGE

Computer Language: It includes wide variety of languages used to communicate with the computers.

Computer languages can be divided into two types:

1) Low level languages (system understandable)
2) High level languages (human understandable)

Q) What is a Programming Language?

- It is an artificial language mainly used to control the behavior of a machine, particularly a computer.
- Programming languages are sub set of computer languages.

Low level languages are further divided into Machine language and Assembly language.

Machine Language:

- This language is directly understood by the computer.
- It doesn't need any translator.
- Programs developed using M.L consists only 0's and 1's.
- It is efficient for the computer and inefficient for the programmer.
- It is difficult to debug programs written in this language.
- The programmer has to know the details about hardware to write a program using this language.

Assembly Language:

- It is considered to be second generation language.
- Unlike machine level language here we use come codes known as mnemonic codes to develop a program.
- But a translator program is required to translate assembly language to machine language. This translator program is called "Assembler".

Disadvantages:

- One of the major disadvantages is that assembly language is machine dependent. A program written for one computer might not run in other computers with different hardware configuration.

HIGH LEVEL LANGUAGES:

- Higher level languages are simple languages that use English and mathematical symbols like +, -, %, / etc. for its program construction.
- Higher level languages are problem-oriented languages because the instructions are suitable for solving a particular problem.
- For EX: COBOL is suitable for business applications, FORTRAN is for scientific purpose.
- These are very easy to learn and use.

Compiler: it is a computer program that translates one text written in computer language to another computer language.

- Original code is known as source code.
- Output or target code is known as object code.

Interpreter:

- It is a type of program translator used for translating higher level language into machine language. It takes one statement of higher level languages, translate it into machine language and immediately execute it.
- Translation and execution are carried out for each statement. It differs from compiler, which translate the entire source program into machine code and does involve in its execution.
- The advantage of interpreter compared to compiler is its fast response to changes in source program.
- It eliminates the need for a separate compilation after changes to each program.
- Interpreters are easy to write and do not require large memory in computer.
- The disadvantage of interpreter is that it is time consuming.
- Thus compiled machine language program runs much faster than an interpreted program.

PAWAN

Software: It is a collection of computer programs.

System software: It is computer software which manages and controls the hardware

Ex: Operating system

Application software: It is computer software designed to perform a specific action directly for the user.

Ex: word processor, paint etc.

Note:

1) Remember it is not possible to run application software without system software

In general system softwares are developed using low level language whereas application softwares are developed using high level languages.

Characteristic Features of C:

- The C compiler combines the capabilities of an assembly language with the features of a high level language.
- So it is useful for developing both system and application software.
- C is highly portable means that C programs written for one computer can be run on another with little or no modification.
- C is well suited for structured programming language.
- The ability of C is , it can extend itself.
- Programs written in C are efficient and fast.
- C is a structured programming language.
- C is case sensitive language (i.e., upper case and lower case characters are recognized differently).
- It is an example for middle level language.

The way how to learn C languages is as follows as:

Character set → Keywords→ Statements→Program
Identifers
Constants

Character Set: The C character set consists of upper and lower case alphabets, digits, special symbols and white spaces.

Alphabets:

A B C ….. X Y Z.

a b c…….. x y z.

Digits: 0 1 2 3 4 5 6 7 8 9

Special Characters:

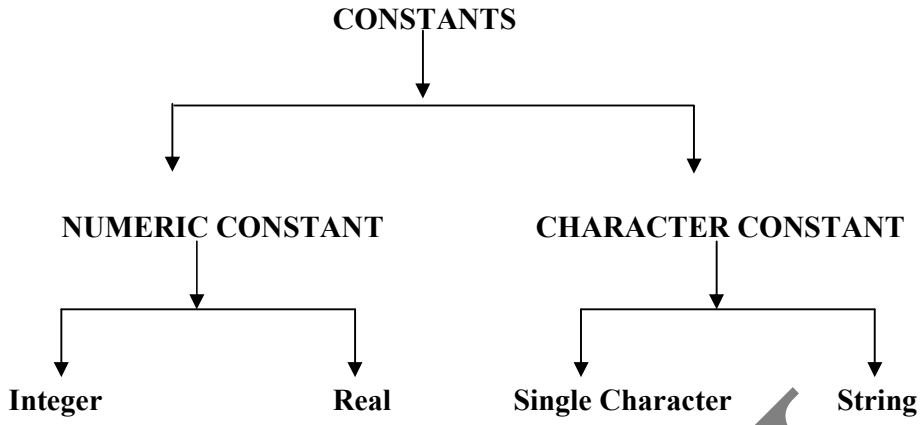
,	**Comma**
“	**Double quotes**
#	**Pound**
*	**Asterisk**
/	**Forward slash**
\	**Backward slash**

White space characters:

Blank space	newline	carriage return
Formfeed	horizontal tab	vertical tab

Constant : Fixed value in the program. In C we have 2 types of constants.

PAVAN



Integer Constant: An arithmetic number without decimal point.

Rules for Constructing Integer Constant

1) It must have at least one digit.
2) It must not contain any decimal part.
3) No special symbols are allowed with in the integer constant.
4) It may be either +ve or – ve.
5) Range : -32768 to 32767

Valid	Invalid
78	10.2,$455,412589
1256	

Types of integers:

a) Decimal Integers: Base is 10. Decimal number can contain digits 0 to 9.
b) Octal Integers: Base is 8.Octal no can contain digits 0 to 7 . Octal no starts with digit zero.
c) HexaDecimal Integer: Base is 16. HDI can contain digits 0 to 9 & alphabets A to F
A -10 B -11 --------- F – 15
Hexa Decimal numbers starts with 0x or 0X

Real Constant : An arithmetic number with decimal point

Rules for Constructing Real Constant

1) It must have atleast one digit.
2) It must contain any decimal part.
3) No special symbols are allowed with in the integer constant.
4) It may be either +ve or – ve.
5) It can't contain more than one decimal point.
6) Range : 3.4 e -38 to 3.4 e +38.

Valid : 3.25,555.1023
Invalid : 9.2.05

Single Character Constant

- In C single character constant must be represented with in the single quote.
- It may be an alphabet, digit ,white space or special character.
- We can't write more than one character in single quotes.
  Ex: 'a' 'A' ' ' '9' '^'
- Character are internally represented by integers **ASCII.**
- (American Standard Code for Information Interchange)
  Ex: 'a' – 97 'A' – 65 '0' – 48 etc.
- There are 256 ASCII values.
- Range : 0 - 255

String Constant: It is a collection of characters represented with in the double quotes.String may contain alphabets, digits, & special characters. String is also known as alphanumeric.
Ex: "rajesh" "134" "a*&5}"

PAVAN

Note: ASCII values not defined for strings whereas each character within the string has ASCII value.

Variable

- It is a named memory location.
- It is a container which can hold some data.
- It can hold a maximum of one value at a time.
- When ever a new value is given to a variable ,its previous value is lost.
- Unlike constant it can change its value.
- In C every variable must be declared before it is used in the program.

Rules for Naming a Variable

1) It must start with an alphabet or underscore (_).
2) Second character onwards it may be alphabet or digit.
3) No spaces, special characters are allowed.
4) It can't be a reserved or keyword.

C words

1) Reserved words (or) keywords.
2) User defined word (or) identifiers.

Reserved Words or keywords:

1) There are 32 keywords in C
2) Each word has a special meaning
3) These words can't be used as user defined words
4) Every keyword must be written in lower case only
5) Keywords serve as building blocks for program statements

Ex: auto, int, char, float etc

User Defined Words Or Identifiers:

1) These are the named elements in the program
2) These words are defined by user.
3) It is used for variable name, function name,--- etc.
4) These words have no meaning.

Tokens: Smallest individual elements or units in the program.

In C we have 6 types of Tokens

1)Keywords 2) identifiers 3) constants 4) Strings 5) Operators 6) special symbols.

DataTypes in C

- Data type specifies the properties of a variable like type of value that it can hold, memory occupied by a variable.
- C supports the following Data Types
  1) Primary or Primitive Data Type
  2) User defined Data Type
  3) Derived Data Type
  4) Empty Data Type

Integer data type:

- An arithmetic number without decimal point.
- It takes 2 byes of memory space.
- Range: -32768 to 32767.
- Keyword: ***int.***

Float data type:

- An arithmetic number with decimal point
- It takes 4 byes of memory space.
- 1st 26 bits are for integer part, next 6 bits are for fractional part.
- Range : 3.4 e -38 to 3.4 e+ 38.
- Keyword: ***float.***

Double data type:

PAVAN

- It is an extension to float.
- It is also known as long float.
- It takes 8 bytes of memory.
- 1st 52 bits are for integer part. Next 12 bits are for fractional part.
- Range : 1.7 e -308 to 1.7 e + 308.
- Keyword: ***double.***

Character data type:

- It takes 1 byte of memory
- Range : -128 to +127
- Keyword : ***char***

Declaration of Variables:

After deciding suitable variable names, we must declare them in our program. Declaration does tow things.

1. it tells the compiler what the variable name is.
2. it specifies what type of data the variable can hold.

The syntax for declaring the variable is as follows:

data-type v1,v2,v3…….. **;**

v1, v2.. are the names of variables. Variables must separated by comma. A declaration statement must end with semicolon. Semicolon acts as a statement terminator.

<u>ASSIGNING VALUES TO VARIABLES USING ASSIGNMENT STATEMENT</u>

Values can be assigned to variables using the assignment operator = as follows:

Variable_Name = constant/variable/expression;

Qualifiers: using which we can modify the behavior of the variable type to which they are applied.

1) Sign (to modify sign behavior) 2) Size (to modify size behavior)
3) Constant 4) Volatile

<u>Sign qualifiers:</u>

1) signed 2) unsigned

The keywords ***signed*** and ***unsigned*** are the two sign qualifiers that specify whether a variable can hold both negative and positive numbers, or only positive numbers. These qualifiers can be applied to the data types ***int*** and ***char*** only

1) **int** is **signed** by default

Unsigned or unsigned int: it takes 2 bytes of memory. There is no sign bit in it all are magnitude bits only.

Range of Unsigned integer: 0 to 65535.

Negative values can't be represented in unsigned data type.

Size Qualifiers: alter the size of basic data type. There are two size qualifiers that can be applied to integers : ***short*** and ***long***

long or long int:

1) It is an extension to int. It takes 4 bytes of memory Space.
2) Range: - 2147483648 to + 2147483647

Note: in most compiler available on DOS , the size of a ***short int*** and ***int*** are the same, both occupying 2 bytes. A ***long int.***

unsigned long: it takes 4 bytes of memory space. All bits are magnitude bits. There is no sign bit

Range: 0 to 4294967295.

long double: it is an extension to double. It takes 10 bytes of memory. First 65 bits for integer part, next 15 bits are for fractional part.

Range : 3.4 e – 4932 to 1.1 e + 4932

const qualifier:

1) ***const*** is a keyword.

PAVAN

2) when ever we want the value of a variable to be constant throughout the program then we declare that variable with this qualifier.
3) when ever we will try to change the value of a variable declared as a constant , in such a case compiler reports compile time error.

volatile qualifier:

1) ***volatile*** is a keyword
2) when we declare any variable with this qualifier it means that the value of this variable may be changed at any time by some external sources

typedef statement:

1) ***typedef*** is a keyword
2) ***typedef*** statement is used to give new names to existing types
ex: **typedef unsignedlong ulong;**
3) From now onwards ***ulong*** will acts as a new type
ulong *u;*
Here u to be of type ulong

printf()

1. It is an output function.
2. It is used to display message and values of variables
3. It is defined in ***"stdio.h" (header) file***
4. It is **call by value** function
5. It deals with monitor

Syntax:

printf("control string",var1,var2,----);

Control string may be a string or format specifier. Var1, Var2 are variables

scanf()

1. It is an input function
2. It is used read some value into memory variables
3. It is defined in "stdio.h" (header) file
4. It is **call by address** function
5. It deals with keyboard

Syntax:

scanf("control string",&var1,&var2,----);

Control string must be format specifier, and ***&(ampersand symbol)*** is necessary before variables. ***&*** means address of. This operator returns the address of variable when it is used with variables. If it is not used unexpected results will come

Block or Compound Statements:

Block is a collection of statements. Block should be enclosed in braces.

```
{
    Declaration statement;
    Executable statement;
}
```

Declaration statements must be placed before executable statements. Declaration statements are processed at compilation time. Executable statements are processed at run time.

int a; **Declaration Statement**
a=10; **executable statement**

PAVAN

Basic Structure of a Program

Documentation section
Link section
Definition section
Global declaration section
main()
{
Declaration Part
Executable Part
}

Subprogram section
FUNCTION 1
FUNCTION 2

- Documentation section contains comment statements
- Link section contains preprocessor directive statements
- Definition section contains symbolic constants definitions
- Global declaration section contains global variables declaration statements
- Every C program must contain ***main*** function
- It is not possible to run a program without ***main***
- ***main*** is a special name it is not user defined word nor reserved word.
- Operating system calls ***main*** function as soon as we run the program.
- ***main*** is user defined function.
- Body of the main function contains declaration as well as executable statements
- Declaration statements can be used to declare local variables to be used in that function
- Sub program section contains userdefined functions
- Function is nothing but group of statements that is designed to perform one specified task

Comment statements:

/* is the beginning of the comment statement
*/ is the end of comment statement
1) Comment statements are ignored by compiler.
2) Source contains comments but not executable object code
3) Comments improve clarity & readability of the programme.
4) We can write comments any where in the programme.
5) Nested comments are not allowed.

/* This is my first program */ *valid comment statement*
/* factorial for /* integer */ number */ *invalid comment statement*

/ Sample Program */*

```
#include<stdio.h>
main()              /* Starting point of program */

{                                /* start of function */


    printf(" This is my first program");/* It deals with monitor */


}                                /* End of the Program */
```

PAVAN

Explanation about Sample Program:

1) any statement that starts with # symbol, is known as **preprocessor directives**
2) the symbol # informs or instructs the preprocessor to insert the contents of stdio.h(standard input and output header file) into our program
3) the contents of the header file become part of the source code when it is compiled
4) the duty of preprocessor to expand macros and header files
5) **main()** is a special function used or called by **OS**
6) **main()** is the starting point of the application
7) every C program must have exactly one **main( )** function
8) if we use more than one **main( )** function the compiler cannot tell which one marks the beginning of the program
9) the empty pair of parenthesis () immediately following **main()** indicates that the function has no arguments
10) the opening **{** brace marks the beginning of the function **main()**
11) closing brace **}** indicates end of main or end of the program
12) all the statements between these two braces form function body
13) the function body contains set of instructions to perform the given task
14) in this case function body contains only one executable statement that is printf statement
15) comment lines are not executable statements
16) printf is a predefined output function for printing output

Steps involved in Executing the Program

1) Editing the Program: In this step using any Editor software we can edit the program and it is saved on a disk with extension .c.

2) Compiling the Program:

- In this step using compiler we can translate source code into machine code i.e., 0's and 1's.
- We can't run a program without compiling it. Computer can understand only machine code hence it is necessary to compile the program.
- Compile the program only once but not every time. i.e., compilation is only once & we can run any no.of times without compiling it again.
- During compilation compiler checks correctness of each statement. If there are any syntactical errors the process of compilation ends here and those errors will be listed out by the Compiler. In this step object code is created with the extension ".obj".

3) Linking the program: In this step all necessary program files and functions are linked to the object code created in step2 and executable object code is created with the extension ".exe". This process is to be done by "Linker" software.

4) Loading or executing the program: In this step executable program is loaded into RAM and address at which program loaded is informed to CPU. Now CPU runs the program from the address given by loader.

- **Alt + F9 → for compiling the program**
- **F9 → for Linking**
- **Ctrl + F9 → for Executing the program**
- **Alt + F5 → to see the output**

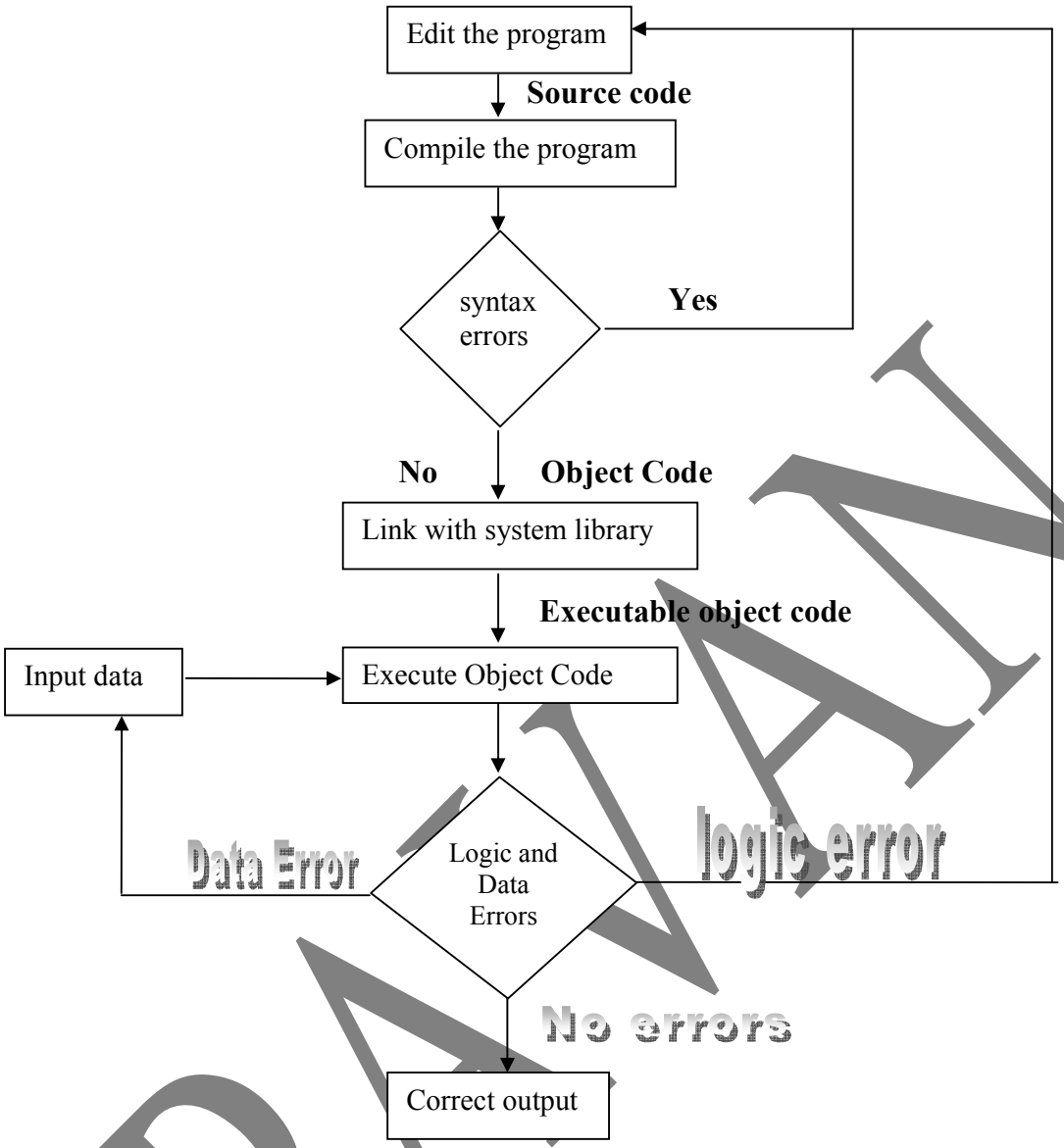
Source Program: It is in **.c** file. It contains source code it is input to the preprocessor. It contains macros & header files.

Expanded Program: it is output of preprocessor & input to compiler. It doesn't contain macros & header files as they are already expanded.

Machine code: It is in ".obj" file. It is output of compiler & is input to linker. It contains 0's & 1's. Functions are not linked.

PAVAN

Executable code: It is in ".exe" file. It is output of Linker and is input to Loader. It contains executable code which is ready to execute



PAVAN

Operator:

- An operator is a symbol that tells the computer to perform certain mathematical or logical manipulations
- Operand is the data upon which operation is carried out
- An expression is a combination of variables, constants and operators written acc to syntax of the language

C supports the following operators

1) Arithmetic operator 2) Relational operator 3) Logical operator 4) Assignment operator 5) Bitwise operator 6) Conditional operator 7) increment and decrement operator 8) Special oerators

Arithmetic Operators:

Ex: +,-,*,/,%

- All the above operators are binary operators. These can operate on any built – in data type
- Integer division truncates any fractional part. The modulo division produces the remainder of an integer division.
- The modulo division operator % cannot be used on floating point data.

Integer Arithmetic:

- It is an arithmetic expression in which both operands are integers.
- Integer arithmetic always yields integer value.

Ex: 3+5*6

Real Arithmetic:

- An arithmetic operation involving only real operands is called real arithmetic.
- Real arithmetic always yields real value.

Ex: 5.0/2.5

Mixed Mode Arithmetic:

- Here the operands are of different type in such a case always lower type is promoted to higher type before any operation is conducted. So result of mixed mode arithmetic is always higher type.

Ex: 9+3.6/2-1.5

Logical Operators: can be used to compare relational expressions.

- && means logical AND.
- || means logical OR.
- ! Means logical NOT.

Logical AND &&

- && operator performs logical multiplication.
- It is a binary operator.
- If atleast one operand is false result is false.

Logical OR ||

- || operator performs logical addition.
- It is a binary operator.
- If atleast one operand is true result is true.

Logical NOT !

- ! Operator performs complement operation.
- It is a unary operator.
- This operator takes single expression and evaluates to true if the expression is false and evaluates to false if the expression is false.
- In other words it just reversed the value of the expression.
- Note: Generally &&,|| operators are used to combine more than one relational expression together.

Relational Operators

- These operators can be used to compare give two quantities.
- Relational expression returns either true or false.
- C supports six relational operators. These operators and their meanings are shown in the following table.

Operator	Meaning
<	is less than
<=	Is less than or equal to
>	is greater than
>=	is greater than or equal to
= =	is equal to
! =	is not equal to

Conditional operator:

- " ?: " is ternary operator.
- Conditional expression takes the following form.

Exp1 ? Exp2 : Exp3

PAVAN

- Exp1 is evaluated first. If it is true then the Exp2 is evaluated and its value becomes the value of the expression. If exp1 is evaluated to false then Exp3 becomes the result of the Expression.

Increment and decrement operators:

- ++ is the symbol for increment operator.
- -- is the symbol for decrement operator.
- Both are unary operators.
- ++ adds 1 to the operand whereas - - subtracts 1.
- These operators manifest in two forms : ***prefix*** and ***postfix.***
- Post increment m++
- pre increment- ++m
- Post decrement m - -
- pre decrement --m
- m++ and ++m mean the same thing when they form statements independently, they behave differently when they are used in expressions on the right hand side of an assignment statement.
- Expansion for m++ or ++m is m = m+1
- Expansion for m-- or --m is m = m-1

a=7 b=++a	a=7 b=a++	a=7 b=--a	a=7 b=a--	a=7 b=-a
a=a+1 b=a	b=a a=a+1	a=a-1 b=a	b=a a=a-1	a=7 b=-7
a=8 b=8	b=7 a=8	a=6 b=6	b=7 a=6	

Assignment Operator:

- It is used to assign the result of an expression to a variable
- C has a set of "shorthand" assignment operators of the form
- v op = exp where **v** is a variable **exp** is an expression **op** is a C binary arithmetic operator
- v op = exp is equivalent to v = v op exp

a+=1	a = a+1
a-=1	a = a-1
a*=n+1	a = a*n+1

Bitwise Operators:

- These operators are used for manipulation of data at bit level
- Bitwise operators may not be applied to float or double
- Except bitwise complement operator all other operators are binary operators
- Bitwise complement operator is a unary operator

PAVAN

Operator	Meaning
&	Bitwise AND
|	Bitwise OR
^	Bitwise exclusive OR
<<	Bitwise leftshift
>>	Bitwise rightshift
~	Bitwise one's complement

1) The result of bitwise OR operation is 1 if atleast one of the operands is 1.

2) The result of bitwise AND operation is 1 if both operands are 1.

3) The result of bitwise EXOR operation is 0 if both operands are 1 or 0 otherwise result is 1.

4) Bitwise left shift operator(<<) can be used to shift the bit level data to left by the specified no.of positions.

5) The effect of shifting a variable's data to left by one bit position is nothing but multiplying the data by 2.

6) Bitwise right shift operator(>>) can be used to shift the bit level data to right by the specified no.of positions.

7) The effect of shifting a variable's data to right by one bit position is nothing but dividing the data by 2.

Special Operators:

- **The comma Operator**
- It can be used to link the related expressions together.
- A comma linked list of expressions are evaluated left to right and the value of right most expression is the value of the combined expression.
- EX: sum = (a =10, b = 15 , a+b);

The sizeof Operator:

- It is a compile time operator.
- It is a unary operator.
- When it is used with an operand it returns the number of bytes the operand occupies.
- The operand may be a variable , a constant or a data type qualifier.
- It can also be used to find the lengths of the arrays and structures.
- It is also used to allocate memory space dynamically to variables during execution of a program.
- ***& and * are pointer operators.***
- ***. and -> are member selection operators.***

Type Casting:

- It is a process of conversion of one data type to another data type.
- These are of 2 types 1) implicit type casting 2) explicit type casting.

Implicit type casting: Here type casting is done by system automatically

Ex: int a = 9.5; here system truncates the decimal part of 9.5 and assigns only integral part to integer variable (TRUNCATION)
float x=10; here system promotes value 10 to 10.0 (PROMOTION).
char ch=65; here integer value 65 is converted into 'A'.

Explicit type casting: Here type casting is to be done by the programmer or user

- The general form of explicit type cast is ***(type – name) expression***
- type – name is one of the standard C data types whereas expression may be a constant, variable or an expression.

Ex: (int) 9.6%(int) 3.2

- Now modulo division is performed in integer mode

PAVAN

Operator precedence and Associativity:
****

- Each operator in C has a precedence associated with it
- This precedence is used to decide how to solve an expression involving
- more the operator
- Generally operators at higher levels of precedence are evaluated first
- The operators of the same precedence are evaluated either from left to right or from right to left depending on the level. This is known as associativity rule
- The following table gives you a complete list of operators, their precedence levels, and their rules of association.
- Rank 1 indicates highest rank and 15 the lowest

operator	Precedence	Precedence	Associativity
( ) **[ ]** **->** **.**	**Function call** **Array subscript** **Member selection operator** **Member selection operator**	**1**	**L→R**
! **~** **+** **-** **++** **--** **&** ***** **sizeof**	**Logical Not** **Bitwise (1's)complement** **Unary plus** **Unary minus** **Increment operator** **Decrement operator** **Address operator** **Indirection operator** **Returns size of data type**	**2**	**R→L**
*** / %**		**3**	**L→R**
+ -		**4**	**L→R**
<< **>>**		**5**	**L→R**
< <= **> >=**		**6**	**L→R**
== **!=**		**7**	**L→R**
&		**8**	**L→R**
^		**9**	**L→R**
|		**10**	**L→R**
&&		**11**	**L→R**
||		**12**	**L→R**
? :		**13**	**R→L**
= **+=** **-=** ***=** **/=**		**14**	**R→L**
,		**15**	**L→R**

Reading a character:

- In C using the following functions we can read a character from the user
- **getchar() getch() getche()**
- getche() and getchar() functions are defined in stdio.h
- getch() is defined in conio.h
- getchar takes the following form

  variable = getchar();

PAVAN

- When the above statement is executed getchar() reads a character from the user and it waits until enter key is pressed and it is echoed on the screen.
- getche() also do the same but it doesn't wait for a key after a character is given and the given character is echoed on the screen.
- getch() is similar to getche() but the given character is not displayed on the screen.

Note: 1) use getch() to read password

2) use getch() at end of every program to see the immediate output

Writing a Character

- Like getchar there is an analogous function putchar for writing characters one at a time to the terminal.
- It takes the form as shown below.

  putchar(variable name);
- Where variable name is a type **char** variable containing a character. This statement displays character contained in the **variable name** at the terminal.

Character Related Functions

- isalpha(ch) – returns true if ch contains alphabet otherwise false.
- isdigit(ch) - returns true if ch contains digit otherwise false.
- isalnum(ch) = isalpha(ch)+isdigit(ch).
- The above function returns true if ch contains alphanumeric otherwise false.
- isspace(ch) – it returns true if ch contains white space character otherwise false.
- islower(ch) – returns true if ch contains lowercase alphabet otherwise false.
- isupper(ch) – returns true if ch contains uppercase alphabet otherwise false.

Character Conversion Functions

- tolower(ch) – converts the given character in to lower case.
- toupper(ch) – converts the given character in to upper case.

Note: All the above functions are defined in **"ctype.h".**

Mathematical Functions

Function name	No.of Arguments	Type Of the argument	Return type	purpose
exp()	1	double	double	To find exponential of X
log()	1	double	double	To find natural logarithm of X
log10()	1	double	double	To find base-10 logarithm of X
pow()	2	double, double	double	To find x to the power of y
sqrt()	1	double	double	Square root of X
ceil()	1	double	double	To find smallest integer not less than X
floor()	1	double	double	To find largest integer not greater than X
fabs()	1	double	double	Absolute value of X
sin()	1	double	double	Sine of x
cos()	1	double	double	cosine of x
tan()	1	double	double	Tangent of x
asin()	1	double	double	Arc Sine of x
acos()	1	double	double	Arc cosine of x
atan()	1	double	double	Arc Tangent of x
sinh()	1	double	double	Hyperbolic Sine of x
cosh()	1	double	double	Hyperbolic cosine of x
tanh()	1	double	double	Hyperbolic Tangent of x

Note: All the above functions are defined in "math.h"

PAVAN

DECISION MAKING AND BRANCHING

C possess the following decision making statements

- **if** statement
- **switch** statement
- **conditional operator**

DIFFERENT FORMS OF IF STATEMENT

- **Simple if statement**
- If – else
- Nested if – else
- else – if ladder

Decision making with if Statement:

1) it is powerful decision making statement.
2) it is used to control the flow of execution of statements.
3) it is basically two-way decision statement.
4) always it is used in conjuction with a condition or expression.
5) computer evaluates the expression and then depending on value of expression it transfers control to a particular statement.

SIMPLE IF STATEMENT

```
if( condition )
{
        statement block;
}
statement x;
```

Statement block may contain one statement or more statements. If the condition is true, statement block will be executed otherwise block statement x will be executed. When the condition is true both statement block and statement x are executed.

if – else statement

```
if ( condition )
{
        True – block statement(s)
}
else
{
        false – block statement(s)
}
```

If the condition is true then true block will be executed otherwise false block will be executed.

Nesting of if – else statement

When a series of decisions are involved, we may have to use more than one if…else statement in nested form as follows:

```
if ( condition 1)
{
    if ( condition 2)
    {
        Block - 1
    }
    else
    {
        Block - II
    }
}
else
{
```

PAVAN

```
        Block - III
    }
```

If conditon1 is false block – III will be executed otherwise it continues to perform second test iff cond-2 is also true, block 1 will be executed otherwise block – II will be executed.

The else if ladder

```
    if(cond 1)
        statement 1
    else if ( cond 2)
        statement 2
    else if ( cond 3)
        statement 3
        ---------
        ---------
    else
        default statement
Statement x;
```

The conditions are evaluated from the top, downwards. As soon as condition is found to be true, statement associated with it is executed and the control is transferred to statement x. When all the n conditions becomes false, then the final else containing the default statement will be executed

The switch Statement

- It is a built in multiway decision statement in C.
- In general switch statement can be used as an alternative to if – else statement.
- switch can be used when we know the exact value of a variable.
- Whereas if – else can be used when we know either exact value or range of values for a variable.
- switch statement tests the value of a given variable or result of the expression against a list of case values and when a match is found a block of statements associated with that case is executed.
- The expression or variable must be an integer or character type.
- Value1, Value2 …. are constants or expressions are known as case labels.
- case labels must end with a colon.
- block – I block – II are the statement blocks and may contain zero or more statements.
- When the switch is executed the value of expression is successively compared against the values. If a case is found whose value matches with the value of the expression then the block of statements that follows the case are executed.
- The break statement signals the end of a particular case.
- When the break statement is executed program control immediately exit from the switch statement.
- The default is an optional case.
- If no match is found in that case default block will be executed if it present, if doesn't present no action takes place.

Syntax:

```
switch(expression)
{
        case value – I :
                block 1
        break;
        case value – II :
                block 2
        break;
        ---------
        ---------
```

PAVAN

```
default:
default block
break;

}
```

The goto Statement

- goto statement breaks the normal sequential execution of a program.
- It can be used to branch unconditionally from one point to another point in the program.
- goto requires label in order to identify the place where the branch is to be made.
- Label is the name given to the statement to where the branch is to be made.
- Name must be C Valid and it must be followed a colon.
- General forms of goto statements.

```
 goto label;                 label:
-------------                statement;
--------------               -------------
--------------               -------------
label:                        --------------
statement;                    goto label;
```

Forward Jump Backward Jump

- Incase of backward jump some loop will be formed and some statements will be executed repeatedly.
- Incase of forward jump some statement will be skipped away from execution.

Note: unconditional backward jump put the program in a permanent loop known as an infinite loop

LOOP CONTROL STATEMENTS

- Looping is a process of repeated execution of block of statements until the condition for its termination is satisfied.
- A loop therefore consists of two segments, one is body of the loop and other one is condition known as control statement.
- A looping generally contains the following 4 steps
  - Setting and initialization of a counter variable
  - Execution of the statements in the loop
  - Test for a specified a condition for execution of the loop
  - Incrementing the counter
- Depending the position of control statement (condition) in the loop a control structure may be classified as either entry controlled loop or exit controlled loop.
- In the entry controlled loop condition is tested before body of loop get executed.
- In exit controlled loop condition is tested at the end of the body of the loop and therefore the body is executed unconditionally for the first time.

The C language provides the following 3 loop constructs for performing loop operations

1. **The while statement**
2. **The do statement**
3. **The for statement**

The while statement

syntax for while

```
while(testconditon)
{
    body of the loop
}
```

- while is an entry controlled loop.
- First condition is evaluated if it is true then body of the loop is executed.

PAVAN

- After executing the body, condition is once again evaluated ,if it is true body is executed once again.
- This process continues as long as condition is true.
- The body of the loop may have one or more statements.

Note: The braces are not required if body contains only one statement.

The do statement

- do-while is exit controlled loop
- In some cases it might be compulsory to execute the body of the loop before the test is performed. Such situations can be handled with the help of the do statement.

```
do
{

    body of the loop

}while(testconditon);
```

Note: since do – while is an exit controlled loop the body of the loop is always executed at least once

The for statement

- It is another entry controlled loop
- General form of the for loop is

```
for ( initialization ; test – condition ; increment/decrement )
{
                body of the loop
```

The execution of the for statement is as follows:

- Initialization of control variables is done first.
- Now the value of the variable is tested using test condition. If the test condition is true body of the loop is executed otherwise loop is terminated.
- After executing the body of the loop, control is transferred to increment/decrement block.
- Test condition is once again evaluated, and if it is true body is executed once again.
- This process continues till value of the control variable fails to satisfy the condition.

Additional Features of for loop

- More than one variable can be initialized at a time in the for statement.
- Like initialization section the increment/decrement section may also have more than one part.
- It is also permissible to use expressions in the initialization and increment sections.
- Both initialization and increment sections are omitted in the for statement.
- Initialization can be done before the for statement and the control variable is incremented. inside the loop. In such cases the sections are left blank. However the semicolons separating the sections must remain.
- Another feature is that the test – condition may have any compound relation and the testing need not be limited only to the loop control variable.

Use of break statement

- When the break statement is executed, control goes out of loop. Statements following break are skipped along with remaining iterations of the loop. Break can be used in for, while, do while loop & switch.
- When the loops are nested, the break would only exit from the loop containing it i.e., the break will exit only a single loop

Use of continue statement

- when continue statement is executed, control goes to next iteration of loop. Statements following continue are skipped. We can use continue statement in for loop , while loop, do – while loop but not in switch. Use continue statement to skip the current iteration

ARRAYS

- Array is an example for Derived Datatype

PAVAN

- An array is group of related data items that share a common name (or)
- An array is collection of homogeneous elements that share a common name

Declaration of Arrays:

datatype Array_Name [size] ;

- Here data type specifies the type of values
- Array_Name is the name given to the array
- Size specifies the max no. of elements that an array can accommodate
  Ex: int a [20] ; Now array **a** can accommodate 20 values of int type
  float avg [10]; This array can store average marks of 10 students
  char ch[30]; here ch is String variable
- Generally pair of square brackets [ ] can be used to represent an array
- Individual values of an array are known as elements
- Memory for entire array should be allocated at the time of compilation ,known as static memory allocation (SMA)
- All the elements of an array will be stored in contiguous memory locations
- A particular element in an array can be indicated by a number called index or subscript with in the pair of square brackets preceded by arrayname.
  X[0] – represent 1st element
  X[i] represent (i+1) th element
- Subscript must be + ve no
- Subscript must be either integer variable or constant or integer expression
- Subscript specifies the no. of elements to be crossed from the beginning
- C doesn't performs bounds checking i.e., no predefined function available in C to check validity of subscript

Dimension Vs Subscript:

- In dealing with independent elements, the number used inside the square brackets is known as subscript. The number used in referring to the whole array as a family is called dimension

Initializing an Array

- We can initialize an array like a variable in the following way

  datatype array_name[size] = { list of values };

- The values in the list must separated by commas
- Ex: int a [5] = { 10,20,30,40,50};
- The no. of values in the initialization list must be less than or equals to size of an array
- If the no. of values in the initialization list are less than the specified size ,then the remaining elements are set to be zero
- If the no. of values in the initialization list are greater than the specified size, then compiler reports the following error message
  - **Too many initializers**
- Size of an array can be omitted during its initialization, in such a case size of an array equals to no. of values in the initialization list

  Ex: float avg[ ]={ 2.3,1.005,9.36};

- In C we can't initialize only selected elements

Two dimensional arrays

- It is a collection of one dimensional arrays
- Generally one dimensional arrays can be used to store row of values whereas 2D arrays can be used to store table of values
- **Declaration of Two dimensional Array:**

  type array_name [size1][size2];

- Size1 specifies no .of rows and size2 specified no. of columns
- Like one-dimensional arrays, two – dimensional arrays can be initialized by the following manner

  type array_name [size1][size2]={ list of values };

PAVAN

- Ex: int M[2][3]={10,20,30,40,50,60};
- In the above case elements of first row are initialized to 10,20,30 and second row elements are initialized to 40,50,60
- Initialization can be down row wise also
- int M[2][3]={ {10,20,30},{40,50,60} };
- We can also initialize a 2D array in the form of a matrix

```
int M[2][3]={
                {10,20,30},
                {40,50,60}
        };
```

- If the values are missing in an initializer, they are automatically set to zero
- The following short cut method is to be used to set all the elements to zero
- int M[3][3]={ {0}, {0}, {0} };
- while initializing an array it necessary to mention column dimension where as row dimension is optional

Base address: Address of the first element in an array is known as Base Address

Q) How to represent a Matrix in C ?

A) Using 2D array

Q) How to represent individual values in 2D array?

A) M[i] [j] where M – array name the above statement represents the element in ith row and jth column in the matrix M or 2D array M

Strings

- Character arrays also referred as Strings
- A String is an array of characters ends with a special symbol known as null character (\0).

Q) How to declare a String?

A) String is not a built in data type like integer.So, we do not have the opportunity to declare a variable straightaway like

```
string svar;
```

Declaration of a String

```
char string_name[ size ];
```

- The size determines the number of characters in the string
- When the compiler assigns a character string to a character array, it automatically appends null character ('\0') to the string.
- So size must be equal to max. no. of characters in the string plus one
- Character array should be initialized when they are declared. C permits a character array to be initialized in either of the following two forms
- char ch[9]={'c','o','m','p','u','t','e','r','\0'};
- char ch[9]="computer";

Note:

1. when we initialize a character array by listing its elements, we must supply explicitly the null character
2. C also permits us to initialize a character array without specifying the size of array. In such cases, the size of the array will be determined automatically, based on the number of elements initialized.
3. For ex: char ch[ ]="computer"

Reading Strings from Terminal:

- scanf function can be used with format %s to read a string
  - char name[25];
  - scanf("%s",name);
- & can't be used before character array name while reading a string using scanf function
- scanf terminates reading the input when it finds white space character

PAVAN

- To overcome this problem C provides one more function to read a string that is **gets** function which is defined in stdio.h
    - **gets(string-name);**
- The above statement will read a group of characters into the string variable
- So scanf function can be used to read a word whereas gets function can be used to read a line of text

Writing strings to screen

➢ We use either printf function with %s format or puts function to display the string on monitor

Ex: printf("%s",string-name);
puts(string-name);

Note: Both gets and puts functions are unformatted i/o functions

String Handling Functions

All the following string handling functions are defined in string.h file except atoi function

- **strlen() :** This function can be used to count the number of characters in a string excluding null
- **strrev():** This function can be used to reverse the given string
- **strcmp():** This function compares the given two strings, character by character and returns difference in the ASCII codes of the first pair of characters where a mismatch is found
- **strcpy():** This function can be used to copy one string to another string
- **strncpy():** this is used to copy only first **n** characters from source string to destination string
- **strcat():** This function can be used for string concatenation (means appending one string to another string)
- **strncat():** it is used to append only the first n characters from source string to destination string
- **strlwr():** it can be used to convert the given string into lower case
- **strupr():** This function can be used to convert the given string into upper case
- **atoi() :** it can be used to convert the given string from ascii format to integer format and this function is defined in stdlib.h file

Function	No. of Arguments
strlen	1
strrev	1
strlwr	1
strupr	1
atoi	1
strcpy	2
strncpy	3
strcmp	2
strcat	2
strncat	3

Structures

Q) What is a structure?

Structure is a collection of logically related data items of same or different type (or) Structure is collection of heterogeneous or homogeneous elements.

Q) Why a Structure?

To create user defined data type

Q) What is the use of structure?

PAVAN

A) Using structure we can represent one business entity information in a flexible manner
There are 2 parts of structure
1) Structure definition 2) Structure declaration

Structure definition: We can't create a structure without defining it

- The general format of Structure definition is as follows:

```
struct tag_name
{
        data_type member1;
        data_type member2;
        -------------------------
        -------------------------
};
```

- **struct** is a keyword that can be used to declare a structure
- tag_name is the name of the structure and is called as structure tag
- Rules for naming a structure is same as rules for naming a variable
- With in the program no two structures can have the same tag or name
- member1, member2 are known as structure elements or members
- Each member may belong to a different type of data
- Structure definition must ends with semicolon

Ex:

```
struct Employee
{
        int empno;
        char ename[24];
        float salary;
};
```

The above structure definition simply creates a format known as template but it doesn't occupy any memory

- We can't initialize members of the structure in structure definition as no memory is allocated to members

Q) Where to declare a Structure?

A) Generally structure definitions appear at the beginning of the program. They may also appear before the main, with in the main or in any function

Structure declaration:

- We can't declare a structure without defining it. Define the structure first followed by declaration
- struct tag_name var1,var2,---;
- **Ex: struct** Employee e1,e2;
- Now **e1** is a variable of type struct employee and is known as structure variable
- Structure variables also referred as objects
- Memory is allocated to members of the structure only when the structure is declared but not when it is defined
- Size of a structure equals to sum of sizes of members of a structure
- sizeof(struct employee) = 2 + 24 + 4 bytes
- structure **e1**contains garbage values as it is not initialized
- It is allowed to combine both structure definition and its declaration in one statement as

```
struct Employee
{
        int empno;
        char ename[24];
        float salary;
}e1;
```

Q) How to access the members of a structure?

Using member selection operators

- **.** (dot or period operator) it can be used in case of structure variable
- ->(arrow operator) it can be used incase of structure pointer

PAVAN

structure variable . Member of a structure

e1**.**empno

e1**.**salary

Giving values to the members of a structure:

- We can assign values to members of a structure in a numbers of ways

Ex: e1.empno = 101;

e1.salary = 2555;

- We can also use scanf function to read values in to the members of a structure

```
scanf("%s",e1.name);
scanf("%d",&e1.empno);
```

Structure initialization:

- structure variable can be initialized like a normal variable
- **struct** Employee e1={101,8520,"ramu"};
- type, Order of the values must match with that of members in the structure definition
- Now e1.empno is 101 e1.salary is 8520 e1.ename is ramu
- **struct** Employee e2={"ramu",8520,101}; /* error */
- **struct** Employee e3;
- e3={"ramu",8520,101}; /* error */
- A structure must be initialized at the time of its declaration only
- We can initialize the structure in the following way also

```
struct
{
        int x;
        char ch;
        float n;
}s={10,'l',2.3};
```

Note: structure tag is optional in this case

- Using assignment operator we can copy the data stored in one structure variable to another structure variable

```
struct Employee e1={101,8520,"pavan"};
struct Employee e2;
e2=e1;
```

- But however both structures or structure variables must be of same type

Q) Is it possible to declare an array with in the structure?

A) Yes, C permits us to declare arrays within the structure. It may be either single or multidimensional array of any type

Q) What is Nested structure?

1) Defining one structure within another structure is known as nesting of structure

2) Declaring one structure variable(object) as a member of another structure is also known as nesting of structures

Note: A structure must not contain the object of same structure as its member

Q) How to access members of a inner structure?

A) They can be accessed by chaining all the concerned structure variables (from outer most to inner – most) with the member, using dot operator.

Array of structures: if we want to represent **n** no. of students , employees or books information in such case, instead of declaring **n** no. of structure variables better to declare an array of structures

Q) How to declare array of structures?

A) **struct** tag_name Array_Name [size] ;

Ex: **struct** Employee e[20];

Now we can store 20 employees information in the above array

struct Student s[100];

Here s[0] ---- 1st student information

s[1] ---- 2nd student information

PAVAN

---
s[n-1] ---- nth student information

Array of structures can be initialized in either of the following two ways

```
struct Student
{
        int rno;
        char name[25];
}s[3]={ {101,"abc"},{102,"xyz"},,{103,"klm"} };
```

or

```
struct Student s[3]={ {101,"abc"},{102,"xyz"},,{103,"klm"} };
```

Pointer to the Structure:

1) We can create a pointer to a structure (user defined data type)

struct tagname *pointername;

Ex: struct student *ptr;

Here ptr is a pointer to a variable of type **struct** student

2) if we have a pointer to a structure then we can access the members of a structure using arrow or indirection operator represented with the symbol →

Ex: ptr → rno

ptr → marks

3) A structure can contain pointer to the same structure as its member

4) A structure which contains pointer to the same structure as its member is referred as Self referential structure

UNIONS

- Union is a data type in C
- In terms of declaration syntax, **union** is similar to a **structure**
- The only change in the declaration is the substitution of the keyword **union** for the keyword **struct**
- **Union** may contain members of different data type
- The major difference between them is in terms of storage
- In structures, each member has its own memory location whereas in union all members can share a common memory location
- The compiler will allocate sufficient storage for the union variables to accommodate the largest element in the union
- Other elements of the union use the same space, so it can handle only one member at a time
- When a different member is assigned a new value, the new value overwrites previous value
- So only that member which is last written, can be read.
- To access a union member, we can use the same syntax that we use for structure members

FUNCTIONS

Definition: The block of statements grouped under one common name which performs one specified task or function is self contained block of code that performs a predefined task or job or work

- Program is a collection of functions whereas function is a collection of statements.
- Each function in the program performs a different task.
- Every C Program must contain main function

PAVAN

- Functions of the program can be defined in any order
- Order in which functions are called matters but the order in which they are defined doesn't matter.
- Execution of the program always starts with main function, irrespective of where it is defined in the program
- We can call any function except main function
- main function called by **Operating system** implicitly

OS
↓
main
↓
f1()
↓
f2()
↓
f3()

Note: Function which is called first that is terminated last. Here functions are terminated in the order of f3(), f2(), f1(), main()

Type of functions:

1) **Predefined functions:** These functions are already defined just call(use) the function. But before using any predefined function include an appropriate header file **(.h file)** which contains prototype of the functions.
   Ex: printf(), scanf(), pow(), clrscr()
2) **Userdefined functions:** our own defined functions are known as userdefined functions.

Function definition: specifying the task to be performed by the function or defining the body of the function which performs a task is known as Function definition

syntax:

```
returntype Functionname(Declaration of parameters (formal))
{
        Declaration of local variables;
        Statement 1;
        Statement 2;
        ----------------
        ----------------
        return (value); or simply return
}
```

Return type: it specifies the type of the value that a function returns. If a functions returns integer its return type is simply **int** , if returns nothing its return type is **void.** If the return type of any function is void it indicates that function performs some action but returns nothing.

Ex: clrscr() performs action but returns nothing

```
void f1( )
{
        return 12.3                    /* error */
}
```

We can't return a value from void function

Note: If return type is not specified default return type is **int**

```
f1( )
{
        return(10.8);
}
```

The above function returns only **10** since default return type is int. So float value is type casted to int

PAVAN

Function name: Every function has a name. A function is identified by its name only.
While naming a function the following rules must be followed

1. No two functions can have same name. function name must be unique
2. function name must be userdefined world but not reserved word(keyword)
3. function name and variable name better not to be the same(it is convention but not compulsion)
4. special symbols(except underscore _) and spaces are not followed with in the function name
5. function name must start with an alphabet or underscore

Function heading: consists of 3 parts
a) return type b) function name c) declaration of parameters
EX:

```
int max(int x, int y, char ch) /* function heading */
{
        Body of the function
}
```

Note: There is no semicolon at the end of the function heading as it is not a statement.
Parameter: it is the input supplied to the function. A function may have zero or more parameters that depend upon task to be performed by the function. To find largest of two numbers, we must supply two parameters to that function.
Ex: clrscr has no parameters, where as sqrt has one parameter, pow has two parameters
Types of parameters:

1) **Actual parameter(AP)**
2) **Formal parameter(FP);**

Actual parameter:

- These are parameters used in the function call (or) input supplied to the function call
- When a function is called value of AP is get copied to FP on one to one basis
- AP may be a value , variable or expression
- The type of AP is type casted to type of FP
- AP & FP may have same name

Formal parameter:

- These are the parameters used(declared) in the function heading
- FP always must be a variable
- Formal parameters must be declared individually
- Formal parameters can't be initialized explicitly in the function heading they are implicitly initialized as soon as function is called

Actual & Formal parameters must match in
1) order 2) type 3) number

<u>return statement</u>

- **return** is a keyword
- It can be used to send or copy a value from called function to calling function
- A function can return one value or none but it is not possible to return more than one value from the function

Syntax:

```
return(value/variable/expression);
        Or
return
```

- Generally return is the last statement in the function
- Statement block following return statement are skipped away from the execution
- Control comes out of the function as soon as return statement is executed
- A function can contain more than one return statement but only one among them is executed
- When a function is returning some value, the type of value is type casted when the type of return value is different from return type declared in function heading

PAVAN

- If a function returns a string its return type is char*

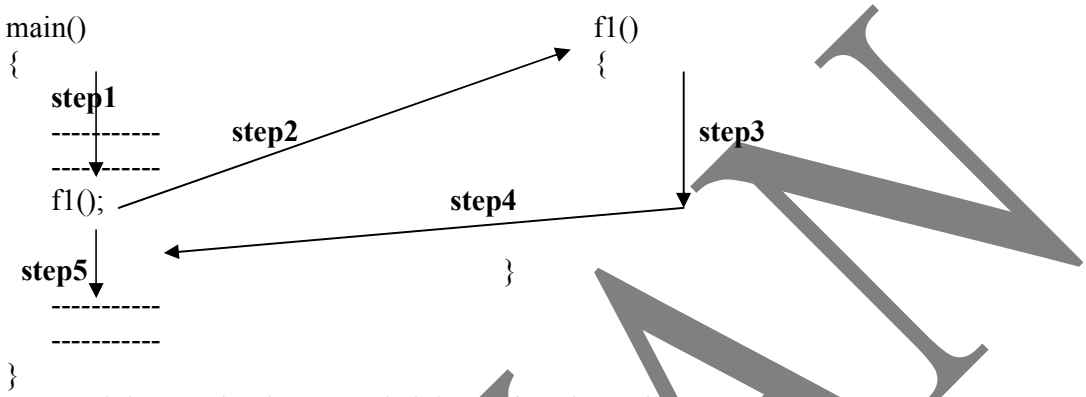
Ex: return("HYD"); here "HYD" is represented some where in the memory, and address of the first character is returned

Calling Function: this is the function which calls other function

Called function: this is the function which is called by some other function

Function call or invoking a function

- A function is called simply by its name
- Any function can 't be executed on its own it must be called by some other function
- When a function call is made program control or control of execution branches(transfers) to the called function from calling function
- Once all the statements in the called function get executed control returns to calling function



- Sequential execution is suspended due to function call
- Statements following the function call is executed only after called function execution is over
- We can't define a function with in the another function
- A function call any other function for any no.of times
- A function can call itself also known as **Recursion**

Function Prototype or function declaration:

- Generally a variable must be declared before it is used in the program in the same way a function must be declared before it is called or used
- Function prototype is nothing but its declaration
- Function prototype informs system some details about the function like function name, type of parameters, and what it returns
- Remember memory is allocated when a variable is declared but no memory is allocated when a function is declared.

Ex: int max(int x,int y); /* function declaration or prototype*/

Or

int max(int ,int);

Here max is a function with two integer parameters and returns an integer value. In the function prototype parameter names are optional. Prototype must be same as function heading.

```
int a;    /* variable declaration */
int f1(); /* function declaration */
```

Case1: if the called function is defined after calling function prototype is compulsory for the called function. In this case with out writing prototype if u make a function call compiler reports an error

Case 2: If called function is defined before calling function prototype is optional.

Category of functions

1) function with no argument and return type Ex: clrscr
2) function with no argument but return type Ex: getchar
3) function with argument but no return type Ex: strrev
4) function with argument and return type Ex: strcmp

Advantages of Functions

1) Repetitive code can be avoided
2) Code reusability

PAVAN

3) Modularity

Parameter passing mechanism

1) **call by value (or) pass by value**: sending the value to a function is call by value. Changes made to formal parameters are not reflected to Actual parameters. So when a value is sent to a function it can't be modified in the called function
   Ex: printf function is an example for call by value
2) **call by address:** sending the address to a function is call by address. Here formal parameter must be a pointer. In this case changes made to formal parameters are reflected to actual parameters
   Ex: scanf function is an example for call by address

Passing array as an argument to a function

- There is no call by value mechanism for array
- Array supports only call by address mechanism because sending array is nothing but sending the base address as array name represents the base address
- So what ever modifications made to the formal parameter in the called function it is reflected to original parameter (ie Actual Parameter)
- When ever array passed as an argument to a function formal parameter may be pointer to that array or it may be an array itself
- If formal parameter is an array, then we need not to specify the size of the array
- If it is a 2D array column size must be specified for formal parameter
- But in the function prototype or function declaration we need not to specify row & column size

Passing structure as an argument to a function:

There are 3 methods by which the values of a structure can be transferred from one function to another.

First method: passing each member of a structure as an actual argument to the function call. The actual arguments are then treated independently like ordinary variables

Second Method: passing a copy of the entire structure to the called function

Note: In the above two methods any changes made to structure members within the called function are not reflected in the original structure

Third method: it employs concept called **pointers** to pass the structure as an argument. In this case address of structure is passed to the called function. So in this case called function can access indirectly the entire original structure. This method is more efficient method when compare to other two methods.

POINTERS

Address:

1) Every variable has an address
2) It is OS duty to assign an address to a variable as soon as a variable is declared
3) Generally we make use of &(ampersand) symbol to know the address of a variable
4) We use %u to print an address
5) There is no chance of negative addresses in the memory
6) We can't change the address of a variable.
7) Address remains fixed throughout the variable's life
8) Addresses are represented in hexadecimal format

Generally variables are of two types

1) **Normal variables or variables:** These variables can hold value but not address
   Ex: int x=23; char ch='A';
   int p=&x; /* error */
   We can't store the address of x in p as it is not a pointer. P can hold a value but not address

2) Pointer Variable or pointer: it can hold address of a variable but not value. **If a variable contains address it is known as pointer variable.**

Pointer Declaration:

datatype *pointername;

* is compulsory in the pointer declaration

Ex: int *p;

PAVAN

char *cp; Read all these statement from right to left
float *fp;

P is a pointer to an integer, cp is a pointer to a character, fp is a pointer to float

Note:

1) Size of any pointer is 2 bytes
2) int *p=1000; /* error */
   Pointer can't hold a value. System interprets 1000 as a value but not address
3) int *x,*y; here both x and y are pointers
4) int *x,y; here x is pointer but not y. So * is only for first variable x but not for remaining variables
5) int *p;
   float x=10.2;
   p=&x; /* error */ because an integer pointer can't point to float variable & vice versa.

We can create a Pointer to

1) a variable
2) an array
3) a string
4) a structure
5) a file

Pointer to a Variable:

int a=25; int *p;
p=&a; (or) int *p=&a;

Indirection Operator(*):

- * means value at address
- *p means value at address i.e., 25
- & is address operator * is value operator
- These two operators are unary operators

Prove that *p =a

p = &a

Apply * on both sides we get

*p = *&a (assume address of a is 1000)
= *1000 (means value at address 1000 which is 25)
=25
=a

So *p = a

<u>Different form of * operator</u>

1) int *p ; Here * is used to indicate p is a pointer
2) printf("%d",*p) here * is used as a indirection operator which gives value at address
3) 10*20 here * is multiplication operator
4) scanf("%d%*d%d",&a,&b,&c); here * indicates ignore second input given by the user

<u>Multiple Pointers to a Variable:</u>

float m=10.85;
float *p1, *p2, *p3;
p1=p2=p3=&m;

Here address of variable is stored in p1, p2, p3. They are pointers to same variable m. so a variable can have more than one pointer

Note: but a pointer can't point to more than one variable at a time

<u>Pointer to Pointer</u>

int a=25; int *b=&a; int **c=&b;

PAVAN

Here c is a pointer to pointer.

void pointer in certain cases we need a pointer which can point to any type of data such type of pointer we may call it as void pointer or generic pointer

syntax: void *ptr;

NULL pointer: if a pointer contains NULL value it means that pointer points to no variable. NULL is a predefined constant in stdio.h

#define NULL 0

So a pointer can hold either address or NULL

Operations on Pointers:

1) No arithmetic operation is possible on pointers except subtraction. We can subtract a pointer from another pointer (if and only if both pointers pointing to same array) which gives no.of elements in between the two addresses

2) we can use ++ and - - on pointers

Ptr++ moves pointer to next address

Ptr - - moves pointer to preceding address

When we increment/decrement a pointer its value is incremented/ decremented by the length of the data type that it points to. This length is called **scale factor**

3) We can add an integer or subtract an integer to/from a pointer

4) The expressions such as p1>p2, p1<p2, p1= =p2 are also valid where p1 and p2 are pointers

Pointers and Arrays

When an array is declared, the compiler allocates sufficient memory to hold all the elements of an array in contiguous memory locations and also defines array name as a **constant pointer** to the base address(means address of the first element)

Ex: int x[5]={10,20,30,40,50};

Assume if the address of first element is 1000 now compiler defines array name x as a constant pointer to base address so

printf("%d",*x); output: 10

Explanation: *x means value at address what x contains base address (1000)

So *x means *1000 = 10

Note: &x[i](subscript method) = x+i (pointer method)

x[i] (subscript method) = *(x+i) (pointer method)

so we can access elements of an array using array indexing or pointer method also. But second method is the faster accessing method when compared to array indexing method

suppose if we have an external pointer **p** then we can make the pointer p to point to the array x by the following statement

p=x; or p = &x[0];

Now , we can access every value of an array x using p++ to move from one element to another

Dynamic Memory Allocation(DMA)

1) Allocating the memory as and when required is referred as DMA
2) Allocating the memory at runtime is also known as DMA
3) C provides the following function to get DMA facility

malloc()

calloc()

realloc()

malloc(): This is used to allocate memory dynamically. Syntax is as follows

void *malloc (no. of bytes);

This function expects no. of bytes as an argument, on success it returns a pointer to newly allocated block. So return type is void*. Here the allocated memory is initialized with garbage values. If enough memory doesn't exist for the new block malloc returns NULL.

calloc(): (calculate and allocate)

1) it is same as malloc except that allocated memory is initialized to zero
2) unlike malloc it expects two arguments
3) first argument specifies no.of elements to allocate
4) second argument specifies size of each element

PAVAN

Syntax: void* calloc (no. of elements, no. of bytes);

realloc(): this function can be used to adjust already allocated memory either to increase or decrease.

Syntax: void* realloc(void*, no.of bytes);

free(): free de allocates memory block allocated previously by calloc, malloc or realloc.

Syntax: void free(void *block)

Command Line arguments:

1) arguments supplied at command line, while executing the program from the command line
2) all the command line arguments are internally supplied to main function
3) main function expects two arguments
4) first argument is an integer variable that specifies no. of arguments supplied
5) second argument is an array of character pointers that can store the arguments
6) so the general structure of main function is as follows as

```
void main(int argc, char *argv[])
{

}
```

7) argv[0] contains program name
8) arguments will be stored from index 1 onwards

Types of variables

Local Variable:

1) This is the variable declared with in a function
2) A local variable defined in one function can't be accessed in another function
3) A local variable is lost or removed from memory when the function terminates in which it is defined

Global variable:

1) Global variables are those variables which are declared outside the function
2) A global variable can be accessed in any function in the program
3) Local variable has higher priority over global variable
4) We can't access a global variable when there is a local variable with the same name.
5) A global variable is lost when the program terminates

Advantages of Global Variable:

1) data sharing is possible
2) global variable is visible to all functions

Disadvantages:

1) data is not secure
2) if the value of the global variable is modified by a function accidentally that mistake is carry forward to all other functions

Storage specifiers and scope of a variable

- Scope of the variable refers to the portion of the program in which the variable can be accessed or refer to
- Lifetime of a variable refers to the amount of time that a variable can exist in memory
- Whereas storage specifier specifies the characteristic properties of storage location

1) Automatic Storage class

- **auto** is the keyword used to specify that variable is a automatic variable
- Local variables are **auto(automatic)** by default
- These variables are automatically created and destroyed without our knowledge

Ex: auto int a;

2) static

- In some situations we need to retain the values of local variables even program control comes out of the function
- In that case this keyword is used

Ex: static int x=10;

- Static variable is initialized only for first time but not every time like automatic variable

PAVAN

- If a static variable is declared inside the function it is local static variable if it declared outside the function it is global static variable
- A global static variable can't be accessed in a different file

3) extern

- A global variable can be used from the point of its declaration only
- All the preceding functions of global variable declaration statement can't make use of the global variable
- We can overcome this drawback by writing the following statement in the preceding functions
  extern int x; if x is global variable
  it is request to the compiler to treat x as a global variable even in preceding functions also
- The above statement doesn't allocate any memory to variable x simply it is a request
- extern int x=30; /* error */ because no memory is allocated to variable x
- extern is only for global variables but not for local variables
- This keyword is also used when we want a variable to share among files

4) register

- This keyword is used to interact directly with CPU register
  register int a=10;
  int b=20;
- Here variable **a** is register variable , b is a memory variable
- There are two storage places in the memory
  - Memory
  - Register
- Cpu can access register data faster than memory data.
- Execution becomes fast when more frequently used data is stored in the register
- Register is a part of CPU
- A register variable can hold only integer value but not float
- Memory variables have addresses but not register variables
- So scanf can't be used to read a value into register variables since it has no address
- registers are expensive but memory is cheap

FILES

- a file is a collection of data or it is collection of records
- Each and every file is identified with unique name
- The file name is divided into two parts file name and extension name
- A file name can have maximum of 8 characters and extension name have max 3 characters

When we are working with files the following steps must be followed

Step 1: we need to search whether that file is existed or not
Step2: get the address of that file
Step3: get the permission on that file
Step4: now we can store or retrieve the data
Step5: announce that work with that file is completed

Step 1,2,3 together called file opening
Step4 is called file accessing
Step5 is called file closing.

File Opening:

fopen() is the function which is used to open the given file

FILE * fopen(filename, mode);

Filename and mode are strings

Mode

r – read	r+ read and write
w – write	w+ write and read
a – append	a+ append and read

PAVAN

b- binary mode

- If the mode is specified as read mode the fopen ( ) function returns the address of the file if it exists otherwise it returns NULL
- If the mode is specified as append mode the fopen( ) function returns the address of the file if it exists and existed data is preserved and new data is appended to existed data. If the file doesn't exist it creates a new file
- The fopen( ) returns address of the file so its return type is FILE*

Closing a file:

fclose( ) is a function which announces that , the work with the file is completed

fclose(FILE*);

Q) Why to Close a File?

- Closing the file flushes the buffer (temporary memory)
- Closing the file releases the space taken by FILE structure which is returned by **fopen**
- Operating system imposes a limit on the no. of files that can be opened by a process at a time. Hence closing a file means that another can be opened in its place
- Hence if a particular FILE pointer is not required after a certain point in a program, pass it to fclose and close the file

File accessing:

It involves either storing the data in to a file or retrieving the data from a file. Storing or retrieving can be done in 3 ways

1) Character by character
2) Value by value
3) Block by block

<u>Character by character</u>

getc() or **fgetc() –** functions can be used to read character by character from the given file

Syntax:

char fgetc(FILE*);

Note: This function returns EOF (end of file) character on reaching the end of the file

putc() or **fputc(FILE*)** – functions can be used to print given character to a file

Syntax:

fputc(char, FILE *);

feof() : This function returns TRUE if file pointer points to end of the file otherwise FALSE

Syntax:

feof(FILE*);

Value by Value:

- When a file is accessed character by character then it is not possible to perform arithmetic operations on the data as the data is stored in character format
- To overcome this problem we use value by value method
- fprintf() function can be used to store more than one data in the given file in the given format
- fscanf() function can be used to read data from a file
- While reading the data from a file it must be read in the same format as it is stored
- **getw( )** can be used to read an integer from a file

Syntax:

int getw(FILE *);

- **putw( )** can be used to print an integer to a file

Syntax:

int petw(int,FILE *);

Block by Block:

- **fwrite()** is the function which is used to copy specified no. of blocks from the specified address to the specified file

Syntax:

fwrite(void*,int,int,FILE*);

- first argument specifies the address from which data is to be copied
- second argument specifies block size in term of bytes

PAVAN

- third argument specifies no. of blocks to be copied
- fourth argument specifies file into which data is to be copied
- **fread()** is the function which is used to read specified no. of blocks to the specified address from the specified file

Syntax:

fread(void*,int,int,FILE*);

Note: when ever a file is opened file pointer by default it points to beginning of the file

Now we can also reposition the file pointer using fseek() function

fseek(FILE*, no.of positions, reference)

Reference

0- file beginning 1- current location 2 – file ending

- If the file pointer is to be moved in the forward direction the no.of positions should be specified as +ve value.
- If the file pointer is to be moved in backward direction the no.of positions must be specified as –ve value

fseek(fp,m,1) – it moves file pointer fp by m bytes in forward direction from the current location

fseek(fp,-1,2) – it moves file pointer fp by 1 byte in backward direction from end of file

- **ftell()** is the function which identifies the offset(distance) of the file pointer with respect to the beginning of the file.

Syntax:

long int ftell(FILE*);

- **rewind()** – it can be used to set the file pointer to the beginning

Syntax:

void rewind(FILE*);

ERROR HANDLING DURING I/O OPERATIONS

It is possible that an error may occur during I/O operations on a file. Typical error situations include

- trying to read beyond end - of – file mark
- device overflow
- trying to use a file which is not opened
- trying to open a file which is not existed
- trying to perform an operation on a file, when the file is opened for another type of operation
- attempting to write to a write protected file

The above errors we can handle with feof( ) , ferror( ) functions. ferror( ) function expects file pointer as an argument and returns non zero value if no error detected upto that point, during processing. It returns zero otherwise

int ferror(FILE*);

PAVAN